

Machine Learning

Lecture 8: Deep Learning II

Prof. Dr. Stefan Leutenegger

Smart Robotics Lab
Technische Universität München

Winter term 2023/2024

Roadmap

Structured data

Training deep neural networks

Deep learning frameworks

Modern architectures & tricks

Section 1

Structured data

Different layers

So far we have seen only fully-connected feed-forward layers, and our (deep) NNs were obtained by stacking them.

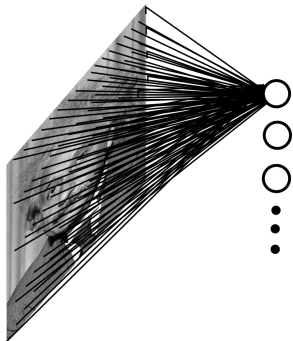
There are many more types of layers for specific tasks/data:

- **Convolution layer** (typically used for images)
- **Recurrent layer** (typically used for sequences; *covered in our MLGS lecture*)
- **Graph convolutional layers** (typically used for graphs; *covered in our MLGS lecture*)
- etc.

These layers leverage the known structure of data and provide an **inductive bias**.

Think of them as building blocks (i.e., lego pieces) that you can compose.

Neural networks for images



From https://cs.nyu.edu/~fergus/tutorials/deep_learning_cvpr12/

We have a 100×100 pixels image and want to process it with a NN with a single hidden layer with 1,000 units.

Problem: In a feed-forward NN this results in **10 million parameters!**

Solution: We can use the convolution operation to build neural networks. This exploits the *high local correlation* of pixel values in natural images.

Example: 1,000 (learnable) 5x5 convolutional filters only require **25,000 parameters**.

CNN: Convolution

Continuous convolution is defined as

$$(x * k)(t) = \int_{-\infty}^{\infty} x(\tau)k(t - \tau) d\tau.$$

This can be intuitively described as a weighted average of the input signal x using the weights (or **kernel**, filter) k at each point in time t .

CNNs use the **discrete** variant:

$$(x * k)(t) = \sum_{\tau=-\infty}^{\infty} x(\tau)k(t - \tau).$$

CNN: Convolution

CNNs for images are based on a **2D convolution**

$$(x * k)(i, j) = \sum_l \sum_m x(l, m) k(i - l, j - m)$$

However, when talking about convolution in CNNs we usually mean *cross-correlation*.

This is what is usually implemented in libraries. It is similar to convolution, but the summation indices are swapped:

$$\hat{x}(i, j) = \sum_{l=1}^L \sum_{m=1}^M x(i + l, j + m) k(l, m)$$

Convolutions usually act on multiple **channels**. Filters therefore have $L \times M \times C_{\text{in}} \times C_{\text{out}}$ parameters.

CNN: Convolution

1 _{x1}	1 _{x0}	1 _{x1}	0	0
0 _{x0}	1 _{x1}	1 _{x0}	1	0
0 _{x1}	0 _{x0}	1 _{x1}	1	1
0	0	1	1	0
0	1	1	0	0

Image

4		

Convolved
Feature

Weights of the sliding window (kernel) are **shared** for every patch.

From http://deeplearning.stanford.edu/wiki/index.php/Feature_extraction_using_convolution

CNN: Convolution



Input

From http://cs.nyu.edu/~fergus/tutorials/deep_learning_cvpr12/

CNN: Padding

Inputs are finite. *What happens at the boundary?*

We can either:

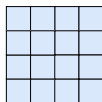
- reduce the output size (by not applying the filter at the boundary)
- **pad** the input (e.g., with zeros, constant values, or by reflecting or repeating the image)

Padding schemes:

- **VALID**: Do not use padding, reduce size in output to $D_{l+1} = (D_l - K) + 1$, where D_l is the input size along a dimension and K is the kernel width (width of its “receptive field”).
- **SAME** (or *half padding*): Add padding so that the input size is preserved, i.e., add $P = \lfloor K/2 \rfloor$ values on each side.
- **FULL**: Add $K - 1$ values on each side, increasing the output size.

CNN: Padding

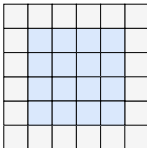
VALID padding



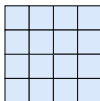
3x3 kernel



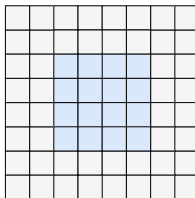
SAME padding



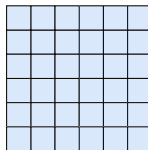
3x3 kernel



FULL padding



3x3 kernel

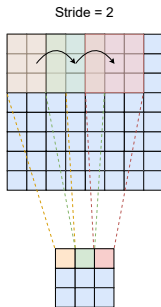
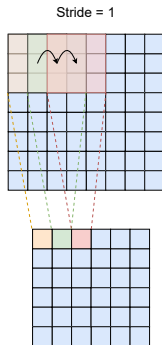


CNN: Strides

A **stride** S is the distance between positions the kernel is applied. Strides $S > 1$ are a way of **downsampling** the signal.

This changes the output size to

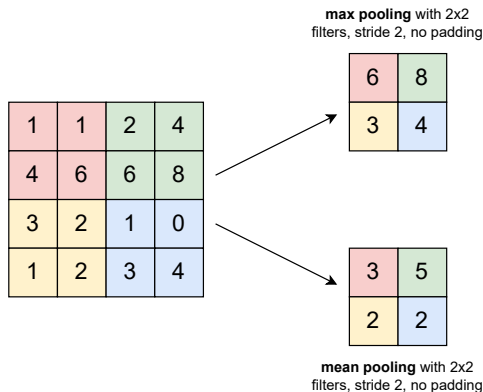
$$D_{l+1} = \left\lfloor \frac{D_l + 2P - K}{S} \right\rfloor + 1$$



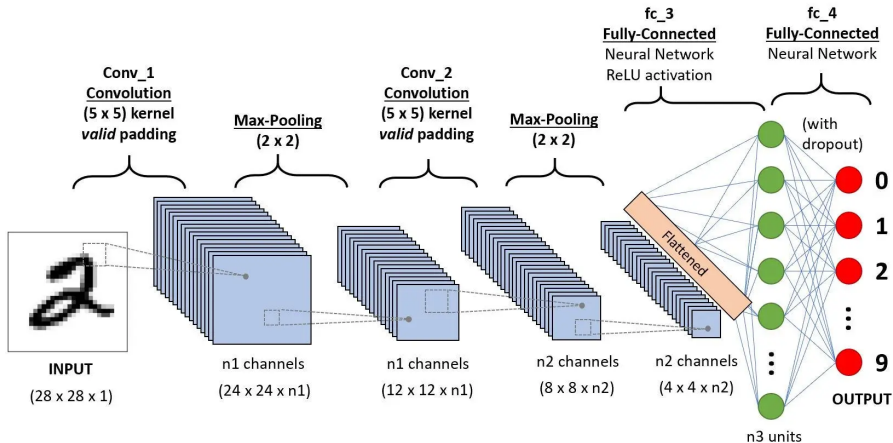
CNN: Pooling

Calculates summary statistics in sliding window. Introduces *invariance* to small movements.

Variants: Max pooling, Mean pooling, L_p norm pooling, etc.



CNN: Convolutional Neural Network

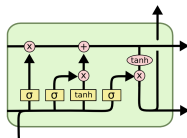
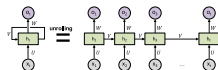


Architectures for other types of structured data

Recurrent neural networks (RNN)

Sequential data

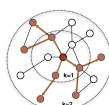
(e.g., text, time series, audio)



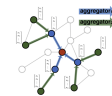
Graph neural networks (GNN)

Graph data

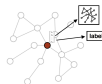
(e.g., social networks, molecules, code)



1. Sample neighborhood



2. Aggregate feature information from neighbors

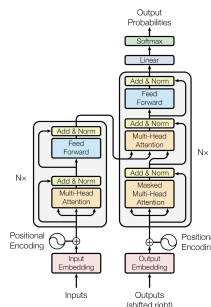


3. Predict graph context and label using aggregated information

Attention-based models

Different types of data

(e.g., images, graphs, sequences, sets)



Images from <https://colah.github.io/posts/2015-08-Understanding-LSTMs/>, Vaswani et al. (2017). Attention is all you need, and Hamilton et al. (2017). Inductive representation learning on large graphs.

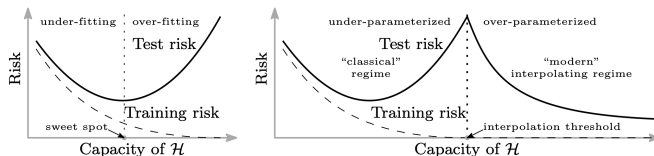
Section 2

Training deep neural networks

Double descent

Bias-variance tradeoff: Models with more parameters have lower bias, but higher variance. Once model complexity passes a certain threshold, the variance dominates and the test error increases again.

Empirical observation: Deep neural networks can be trained to have very low training error, even if they have many more parameters than training examples.



From Belkin et al. Reconciling modern machine learning practice and the bias-variance trade-off. 2019

This behavior is called **double descent** and is not yet fully understood.

Weight initialization

Training a neural network means *changing the weights of a neural network* so that the loss on the training data is minimized.

This is done by *computing the gradient of the loss w.r.t. the weights* and then *updating the weights* via a specific optimization method (e.g., SGD, Adam, AMSgrad, etc.).

Q: from which point do we start optimizing? What initial assignment do we give to the weights?

Weight initialization can be crucial for successful training.

There are 2 essential issues with naïve weight initialization:

- Weight symmetry
- Weight scale

Weight symmetry

If two hidden units have exactly the same bias and exactly the same incoming and outgoing weights, they will always get exactly the same gradient.

Problem: They can never learn to extract different features.

Solution: We break symmetry by initializing the weights to have **small random values**.

Weight scale

If a hidden unit has a **large fan-in**¹, small changes on many of its incoming weights can make the unit saturate (e.g., sigmoid activation function).

If a hidden unit has a **large fan-out**², the gradient can be large and

- cause large changes to the weights during the optimization step
- cause large changes to the weights of the previous layer in the backward pass

Wrong weight scales (mean and variance) can therefore lead to **vanishing** or **exploding gradients**.

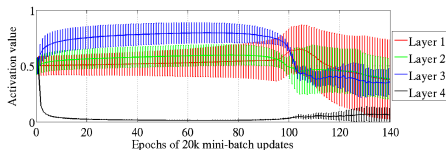
¹Fan-in: number of incoming weights (or number of units in the previous layer).

²Fan-out: number of outgoing weights (or number of units in the subsequent layer).

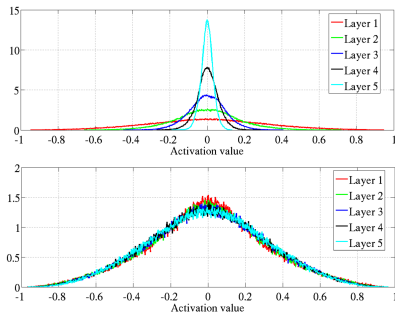
Xavier Glorot initialization

Q: How can we preserve the mean and variance of an incoming i.i.d. signal with zero mean in both forward and backward pass?

A: By controlling the *statistical moments* of the weight distribution.



Mean and standard deviation of activation.
Note saturation of layer 4.



Activation histogram.

Top: Unnormalized. Bottom: Glorot init.

From Glorot, Bengio. Understanding the difficulty of training deep feedforward neural networks. 2010

Xavier Glorot initialization

We can preserve the signal's properties by using weight matrices with zero mean and variance of

$$\text{Var}(\mathbf{W}) = \frac{2}{\text{fan-in} + \text{fan-out}}.$$

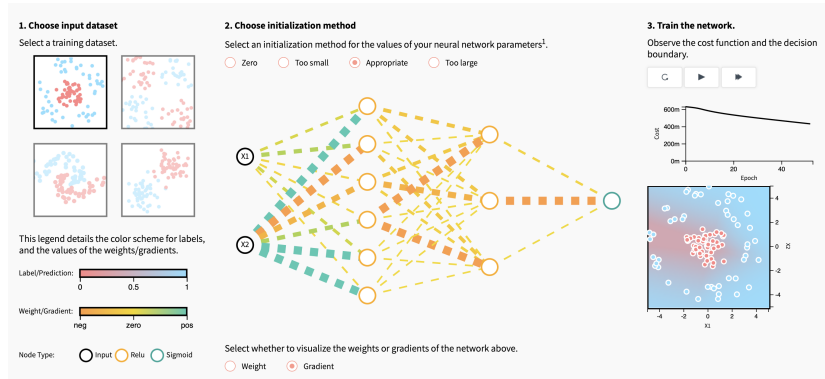
This is **Xavier (or Glorot) initialization**. An example weight distribution fulfilling this is

$$\mathbf{W} \sim \text{Uniform} \left(-\sqrt{\frac{6}{\text{fan-in} + \text{fan-out}}}, \sqrt{\frac{6}{\text{fan-in} + \text{fan-out}}} \right).$$

Note: the output layer for regression tasks is usually best initialized taking into account the target scale.

Weight initialization demo

<https://www.deeplearning.ai/ai-notes/initialization/index.html>



Vanishing and exploding gradients

Deep networks can suffer from the **vanishing** / **exploding** gradient problem

Toy example: We multiply the input t times by a matrix $\mathbf{W}$

$$\mathbf{W} = \mathbf{V} \text{diag}(\mathbf{D}) \mathbf{V}^{-1} \quad \Rightarrow \quad \mathbf{W}^t = \mathbf{V} (\text{diag}(\mathbf{D}))^t \mathbf{V}^{-1}$$

The gradient w.r.t. the elements of $\mathbf{D}$ will likely vanish or explode:

- if $D_{ii} < 1.0$, then D_{ii}^t will be near zero, and gradients will become small — the network will take a long time to converge
- if $D_{ii} > 1.0$, then D_{ii}^t will explode and the computations become unstable

Gradients also vanish when using **saturating** activations (e.g., sigmoid)

Solutions:

- Change the architecture (e.g., use batch norm, change activations)
- Gradient clipping

Regularization

Recall that models with high capacity (like NNs) are prone to **overfitting**. We need **regularization** to prevent this.

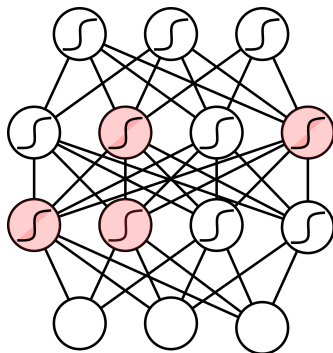
Typically, we use the familiar L_2 parameter norm penalty (or weight decay, which is mostly equivalent). Sometimes we also use L_1 norm to e.g. promote sparsity.

We can combine it with other regularization methods:

- Early stopping
- Dataset augmentation: e.g. rotate/translate/skew/change lighting of images
- Injecting noise
- Parameter tying and sharing
- Dropout

Dropout

Let's look at a neural network with two hidden layers. Each time a training sample is learned, we randomly put to 0 each hidden unit with probability p (usually 0.5).



We are therefore randomly sampling from 2^H different architectures, but these share the same weights.

Hyperparameter optimization

To squeeze every bit of performance out of a NN, one need to do **hyperparameter tuning**:

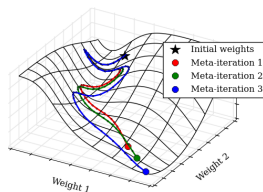
- number of hidden layers (1, 2, 3, ...)
- number of hidden units (50, 100, 200, ...)
- type of activation function (sigmoid, ReLU, Swish, ...)
- optimizer (SGD, Adam, ADADELTA, Rprop, ...)
- learning rate schedule (warmup, decay, cyclic)
- data preprocessing/augmentation
- ...

We often start by finding these by “playing around” with some reasonable estimates. A better way of finding a good set is by using hyperparameter optimization. Random search or Bayesian Optimisation are both viable candidates.

Side note: gradient-based hyperparameter optimization

Q: instead of “searching” for good hyperparameters, couldn’t we **learn** them, e.g., with **gradient descent**?

A: Yes, for *continuous* hyperparameters!



from arxiv.org/pdf/1502.03492.pdf

(An example of) **meta-learning**: backpropagation **through the training procedure** to compute the gradient of the final loss w.r.t. hyperparameters (e.g., *learning rate*). We can then perform a “meta update” on the hyperparameter and repeat.

Few-shot learning: **learn** the initial weights to enable a model to adapt to different tasks with few training examples.

For large neural networks this is however extremely expensive since we have to store all parameters at each training iteration.

Section 3

Deep learning frameworks

Deep learning frameworks

Programming your own NN in Python is quite simple, but not efficient. For efficient code, use one of many open source libraries, e.g.:

- PyTorch
- TensorFlow
- JAX
- MXNet
- ...

You will then find a number of implementations based on such libraries.

Static vs. dynamic computational graphs

Deep learning frameworks build a **computational graph** that defines in which order the operations are performed.

A graph is created on the fly

```
from torch.autograd import Variable

x = Variable(torch.randn(1, 10))
prev_h = Variable(torch.randn(1, 20))
W_h = Variable(torch.randn(20, 20))
W_x = Variable(torch.randn(20, 10))
```



From

<https://medium.com/intuitionmachine/pytorch-dynamic-computational-graphs-and-modular-deep-learning-7e7f89f18d1>

Static vs. dynamic computational graphs

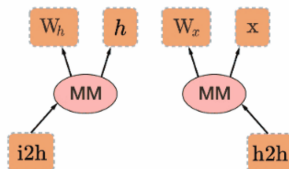
Deep learning frameworks build a **computational graph** that defines in which order the operations are performed.

A graph is created on the fly

```
from torch.autograd import Variable

x = Variable(torch.randn(1, 10))
prev_h = Variable(torch.randn(1, 20))
W_h = Variable(torch.randn(20, 20))
W_x = Variable(torch.randn(20, 10))

h2h = torch.mm(W_x, x.t())
i2h = torch.mm(W_h, prev_h.t())
```



From

<https://medium.com/intuitionmachine/pytorch-dynamic-computational-graphs-and-modular-deep-learning-7e7f89f18d1>

Static vs. dynamic computational graphs

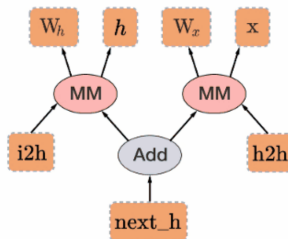
Deep learning frameworks build a **computational graph** that defines in which order the operations are performed.

A graph is created on the fly

```
from torch.autograd import Variable

x = Variable(torch.randn(1, 10))
prev_h = Variable(torch.randn(1, 20))
W_h = Variable(torch.randn(20, 20))
W_x = Variable(torch.randn(20, 10))

h2h = torch.mm(W_x, x.t())
i2h = torch.mm(W_h, prev_h.t())
next_h = i2h + h2h
```



From

<https://medium.com/intuitionmachine/pytorch-dynamic-computational-graphs-and-modular-deep-learning-7e7f89f18d1>

Static vs. dynamic computational graphs

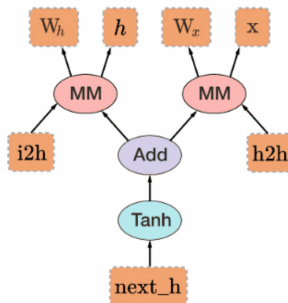
Deep learning frameworks build a **computational graph** that defines in which order the operations are performed.

A graph is created on the fly

```
from torch.autograd import Variable

x = Variable(torch.randn(1, 10))
prev_h = Variable(torch.randn(1, 20))
W_h = Variable(torch.randn(20, 20))
W_x = Variable(torch.randn(20, 10))

h2h = torch.mm(W_x, x.t())
i2h = torch.mm(W_h, prev_h.t())
next_h = i2h + h2h
next_h = next_h.tanh()
```



From <https://medium.com/intuitionmachine/pytorch-dynamic-computational-graphs-and-modular-deep-learning-7e7f89f18d1>

Static vs. dynamic computational graphs

Deep learning frameworks build a **computational graph** that defines in which order the operations are performed.

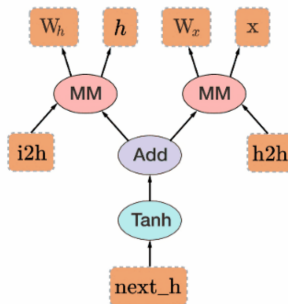
Back-propagation
uses the dynamically built graph

```
from torch.autograd import Variable

x = Variable(torch.randn(1, 10))
prev_h = Variable(torch.randn(1, 20))
W_h = Variable(torch.randn(20, 20))
W_x = Variable(torch.randn(20, 10))

h2h = torch.mm(W_x, x.t())
i2h = torch.mm(W_h, prev_h.t())
next_h = i2h + h2h
next_h = next_h.tanh()

next_h.backward(torch.ones(1, 20))
```



From

<https://medium.com/intuitionmachine/pytorch-dynamic-computational-graphs-and-modular-deep-learning-7e7f89f18d1>

Static vs. dynamic computational graphs

Static variant: we first *define* the computational graph to later *execute* it with actual data (“**Define-and-Run**”).

Dynamic variant: the graph is *defined by executing* the desired operations (“**Define-by-Run**”).

When to use static graphs? Static computational graphs can be optimized by the framework, similar to a compiler optimizing source code. They can then be generated via JIT compilation of annotated functions → **faster execution**.

When to use dynamic graphs? Static graphs cannot be changed at runtime. *Example:* RNNs. With a static computational graph an RNN gets explicitly unrolled for a specified number of time steps. This means that it cannot process sequences of varying time/length. Dynamic graphs can handle this → **more flexible**.

The major frameworks have now moved to dynamic computational graphs, since they are more natural to work with.

Section 4

Modern architectures & tricks

Batch normalization

Goal: accelerate the training of deep neural nets, by reducing **internal covariate shift** → **stabilize** the distribution of each layer's activations.

Idea: introduce a normalization step that fixes the means and variances of layer inputs → **whitening**³ each activation independently.

Too expensive for each training step, use *minibatch* $\mathcal{B}$ statistics instead (or a moving average):

$$\hat{x} = \frac{x - \mathbb{E}_{\mathcal{B}}[x]}{\sqrt{\text{Var}_{\mathcal{B}}[x] + \epsilon}}$$

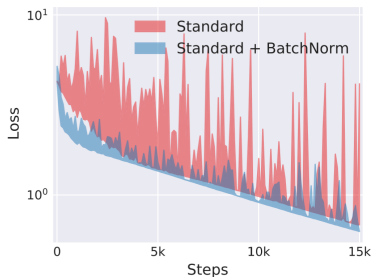
To maintain the model's representational power we also introduce a **scale** parameter γ and an **offset** β . These are learned via backpropagation.

$$y = \gamma \hat{x} + \beta$$

This is done after each layer (before the non-linearity): x is the batch norm layer's input, y its output

³Pre-processing operation on the raw features.

Batch normalization: Why does it work?



Loss landscape and gradient change per optimization step.

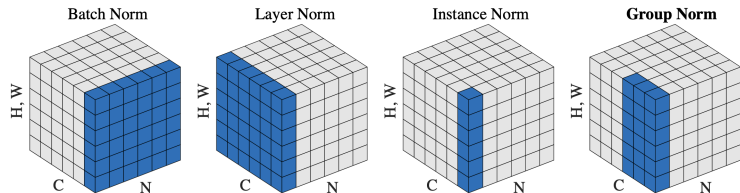
From: Santurkar et al. How Does Batch Normalization Help Optimization? 2018

Batch normalization reduces the dependence of gradients on the scale and initial values of the parameters → **smooth loss landscape**. More predictive and stable gradients help training.

Note: the original explanation based on internal covariate shift turned out to be wrong.

Other normalization methods

- **Layer normalization**: estimates the normalization statistics along the channel dimension (instead of the batch dimension).
- **Instance normalization**: estimates the normalization statistics for an individual sample.
- **Group normalization**: generalizes the batch normalization idea to groups of channels.



N: batch axis, C: channel axis, (H, W): spatial axes.

From: Wu and He. Group Normalization. 2018

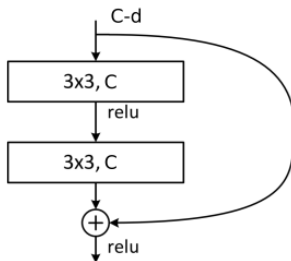
Skip connections

Improve information flow in deep neural networks.

Highway networks: Add layer input to its output, weighted by gate T :

$$\mathbf{y} = f(\mathbf{x}, \mathbf{W})T(\mathbf{x}, \mathbf{W}_T) + \mathbf{x}(1 - T(\mathbf{x}, \mathbf{W}_T))$$

Residual connections: Add previous layers' output to a subsequent layer, skipping multiple (usually 2) layers. Usually no gating or other transformation.



Residual connection. From He et al. Deep Residual Learning for Image Recognition. 2016

Tips and tricks

- Use only **differentiable operations**. Non differentiable operations are e.g., $\arg \max$, sampling from a distribution (*covered in our advanced lecture DGM*)
- Always try to **overfit your model to a single training batch** or sample to make sure it is correctly “wired”.
- Start with small models and **gradually add complexity** while monitoring how the performance improves.
- Be aware of the **properties of activation functions**, e.g., no sigmoid output when doing regression.
- **Monitor** the training procedure and use **early stopping**.
- See also: A Recipe for Training Neural Networks
<https://karpathy.github.io/2019/04/25/recipe/>

Summary

- Convolutional Neural Networks (CNNs):
 - convolution, padding, strides, pooling.
- Training deep neural networks:
 - Xavier Glorot initialization; regularization (dropout); hyperparameter optimization.
- Deep learning frameworks:
 - open-source libraries; static & dynamic computational graphs.
- Modern architectures & tricks:
 - batch normalization; skip connections; tips & tricks.

Reading material

- Goodfellow, Deep Learning: chapters 7, 8, 9, 11
- Karpathy, A Recipe for Training Neural Networks
<https://karpathy.github.io/2019/04/25/recipe/>